[1]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

[2]:
```
df = pd.read_excel(r"C:\Users\Ibhrahim ansari\Desktop\pr-4  message intelligence system(ml)\Message_Intelligence_Dataset_5200.xlsx")
```

Part B: Dataset Understanding & Preparation

[3]:
```
df.head()
```

[3]:

	message_id	message_text	message_length	word_count	num_urls	num_digits	num_special_chars	spam_keyword_score	legit_keyword_score	sender_activity_score
0	900001	Please find the attached invoice for the updat...	99	11	1	4	0	0	1	56.6
1	900002	Let's catch up tomorrow regarding the timeline...	73	12	0	0	0	0	0	16.6
2	900003	Can you send the report by end of day? next Mo...	67	13	0	0	0	0	1	25.7
3	900004	Can you send the report by end of day? 10:30 A...	64	13	0	4	0	0	1	48.8

[4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5200 entries, 0 to 5199
Data columns (total 16 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   message_id              5200 non-null   int64
 1   message_text            5200 non-null   object
 2   message_length          5200 non-null   int64
 3   word_count              5200 non-null   int64
 4   num_urls                5200 non-null   int64
 5   num_digits              5200 non-null   int64
 6   num_special_chars       5200 non-null   int64
 7   spam_keyword_score      5200 non-null   int64
 8   legit_keyword_score     5200 non-null   int64
 9   sender_activity_score   5094 non-null   float64
 10  sender_account_age_days 5087 non-null   float64
 11  messages_sent_last_24h  5038 non-null   float64
 12  timestamp               5200 non-null   datetime64[ns]
 13  hour_of_day             5200 non-null   int64
 14  day_of_week             5200 non-null   int64
 15  spam_label              5200 non-null   int64
dtypes: datetime64[ns](1), float64(3), int64(11), object(1)
memory usage: 650.1+ KB
```

[5]:
```
df.describe()
```

[5]:

_keyword_score	legit_keyword_score	sender_activity_score	sender_account_age_days	messages_sent_last_24h	timestamp	hour_of_day	day_of_week	spam_label
5200.000000	5200.000000	5094.000000	5087.000000	5038.000000	5200	5200.000000	5200.000000	5200.000000
0.246346	0.817692	54.836317	296.671909	8.301906	2025-11-24 03:41:11.538461696	11.523462	3.038846	0.187115
0.000000	0.000000	5.000000	2.000000	0.000000	2025-10-25 01:00:00	0.000000	0.000000	0.000000
0.000000	0.000000	43.100000	148.000000	5.000000	2025-11-09 08:45:00	6.000000	1.000000	0.000000
0.000000	1.000000	54.800000	275.000000	7.000000	2025-11-24 04:00:00	12.000000	3.000000	0.000000
0.000000	1.000000	66.600000	410.500000	10.000000	2025-12-09 02:00:00	17.000000	5.000000	0.000000
3.000000	2.000000	100.000000	1500.000000	38.000000	2025-12-24 00:00:00	23.000000	6.000000	1.000000
0.676798	0.556687	17.697943	191.748754	5.586503	NaN	6.861518	2.062504	0.390042

[6]:
```
df.shape
```

[6]:
```
(5200, 16)
```

[7]:
```
print("-------------------Number of missing values-------------------")
missing_data = df.isna().sum()[df.isna().sum()>0] / len(df)*100
print(missing_data)
```

```
-------------------Number of missing values-------------------
sender_activity_score      2.038462
sender_account_age_days    2.173077
messages_sent_last_24h     3.115385
dtype: float64
```

[8]:
```
print("-------------------Duplicate vlaues-------------------")
df.duplicated().sum()
```

```
-------------------Duplicate vlaues-------------------
```

[8]:
```
np.int64(0)
```

[67]:
```
print("-------------------Fill missing value with knn imputer-------------------")
missing_data_col = ["sender_activity_score", "sender_account_age_days", "messages_sent_last_24h"]
from sklearn.impute import KNNImputer
ki = KNNImputer(n_neighbors=5)
df[missing_data_col] = ki.fit_transform(df[missing_data_col])
```

```
-------------------Fill missing value with knn imputer-------------------
```

[10]:
```
df.drop("timestamp", axis=1, inplace=True)
```

[68]:
```
print("Identify input features and target variable.")
X = df.drop(["message_id", "spam_label"], axis=1)
y = df["spam_label"]
```

```
Identify input features and target variable.
```

[12]:
```
text_col = df["message_text"]
num_col = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
```

[70]:
```
print("Split the dataset into training and testing sets.")
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X,y,
    random_state=32,
    test_size = 0.2,
    stratify=y
)
```

```
Split the dataset into training and testing sets.
```

[14]:
```
print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
print("y_train:", y_train.shape)
print("y_test:", y_test.shape)
```

```
X_train: (4160, 13)
X_test: (1040, 13)
y_train: (4160,)
y_test: (1040,)
```

[71]:
```
print("Scaling")
from sklearn.preprocessing import StandardScaler
ss = StandardScaler()
X_train[num_col] = ss.fit_transform(X_train[num_col])
X_test[num_col] = ss.transform(X_test[num_col])
```

```
Scaling
```

[16]:
```
X_train_text = X_train['message_text']
X_test_text = X_test['message_text']
```

[17]:
```
X_train_num = X_train.drop("message_text", axis=1)
X_test_num = X_test.drop("message_text", axis=1)
```

[72]:
```
before = y.value_counts()
```

[73]:
```
from imblearn.over_sampling import SMOTE
smt = SMOTE(random_state=32)

X_train_resample, y_train_resample = smt.fit_resample(
    X_train_num, y_train
)
```

[78]:
```
after= y_train_resample.value_counts()

print("\n---------\nbefore SMOTE----------""\n",before)
print("\n---------\nafter SMOTE----------""\n",after)
```

```
---------
before SMOTE----------
 spam_label
0    4227
1     973
Name: count, dtype: int64

---------
after SMOTE----------
 spam_label
0    3382
1    3382
Name: count, dtype: int64
```

Part C: Baseline Model - K-Nearest Neighbors

[50]:
```
from sklearn.neighbors import KNeighborsClassifier
knn_class = KNeighborsClassifier(n_neighbors=5)
knn_class.fit(X_train_resample, y_train_resample)
y_pred = knn_class.predict(X_test_num)
```

[51]:
```
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix, f1_score, precision_score, recall_score

print("Accuracy", accuracy_score(y_test,y_pred))
print("\nClassification report:", classification_report(y_test, y_pred))
print("\nConfusion matrix:", confusion_matrix(y_test, y_pred))
```

```
Accuracy 1.0

Classification report:               precision    recall  f1-score   support

           0       1.00      1.00      1.00       845
           1       1.00      1.00      1.00       195

    accuracy                           1.00      1040
   macro avg       1.00      1.00      1.00      1040
weighted avg       1.00      1.00      1.00      1040


Confusion matrix: [[845   0]
 [  0 195]]
```

[52]:
```
for k in [3,5,7,9,11]:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train_resample, y_train_resample)

    y_pred = knn.predict(X_test_num)

    print("k",k)
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("f1_score:", f1_score(y_test, y_pred))
```

```
k 3
Accuracy: 1.0
f1_score: 1.0
k 5
Accuracy: 1.0
f1_score: 1.0
k 7
Accuracy: 1.0
f1_score: 1.0
k 9
Accuracy: 1.0
f1_score: 1.0
k 11
Accuracy: 1.0
f1_score: 1.0
```

[53]:
```
for metric in["euclidean", "manhattan", "minkowski"]:
    knn = KNeighborsClassifier(n_neighbors=5, metric=metric)
    knn.fit(X_train_resample, y_train_resample)

    y_pred = knn.predict(X_test_num)

    print("Metric:", metric)
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("f1_score:", f1_score(y_test, y_pred))
```

```
Metric: euclidean
Accuracy: 1.0
f1_score: 1.0
Metric: manhattan
Accuracy: 1.0
f1_score: 1.0
Metric: minkowski
Accuracy: 1.0
f1_score: 1.0
```

Part D: Support Vector Machine Classifier

[60]:
```
from sklearn.svm import SVC

svm_rbf = SVC(kernel="rbf")
svm_rbf.fit(X_train_resample, y_train_resample)
y_pred_svm = svm.predict(X_test_num)

print("Accuracy Score:", accuracy_score(y_test, y_pred))
print("Classification report:", classification_report(y_test, y_pred))
```

```
Accuracy Score: 1.0
Classification report:               precision    recall  f1-score   support

           0       1.00      1.00      1.00       845
           1       1.00      1.00      1.00       195

    accuracy                           1.00      1040
   macro avg       1.00      1.00      1.00      1040
weighted avg       1.00      1.00      1.00      1040
```

[63]:
```
svm_linear = SVC(kernel="linear")
svm_linear.fit(X_train_resample, y_train_resample)
y_pred_svm = svm.predict(X_test_num)

print("Accuracy:", accuracy_score(y_test, y_pred))
print("Classification report:", classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
Classification report:               precision    recall  f1-score   support

           0       1.00      1.00      1.00       845
           1       1.00      1.00      1.00       195

    accuracy                           1.00      1040
   macro avg       1.00      1.00      1.00      1040
weighted avg       1.00      1.00      1.00      1040
```

Part E: Naive Bayes Classifier & Probability

[56]:
```
from sklearn.naive_bayes import GaussianNB
nb = GaussianNB()
nb.fit(X_train_resample, y_train_resample)
nb_pred = nb.predict(X_test_num)

print("Accuracy:",accuracy_score(y_test, y_pred))
print("Classification report:", classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
Classification report:               precision    recall  f1-score   support

           0       1.00      1.00      1.00       845
           1       1.00      1.00      1.00       195

    accuracy                           1.00      1040
   macro avg       1.00      1.00      1.00      1040
weighted avg       1.00      1.00      1.00      1040
```

[57]:
```
# Conditional probability of spam for each keyword score
for score in [0, 1, 2, 3]:
    data = df[df["spam_keyword_score"] == score]
    prob = (data["spam_label"] == 1).mean()
    print(f"Score {score}: P(Spam | Score) = {prob:.3f}")
```

```
Score 0: P(Spam | Score) = 0.051
Score 1: P(Spam | Score) = 1.000
Score 2: P(Spam | Score) = 1.000
Score 3: P(Spam | Score) = 1.000
```

Part F: Model Comparison & Evaluation

[64]:
```
for name, model in[("KNN",knn), ("svm_rbf",svm_rbf),
                   ("svm_linear",svm_linear), ("gausian_nb",nb)]:
    y_pred = model.predict(X_test_num)
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Precision:", precision_score(y_test, y_pred))
    print("f1_score:", f1_score(y_test, y_pred))
    print("recall_score:", recall_score(y_test, y_pred))
```

```
Accuracy: 1.0
Precision: 1.0
f1_score: 1.0
recall_score: 1.0
Accuracy: 1.0
Precision: 1.0
f1_score: 1.0
recall_score: 1.0
Accuracy: 1.0
Precision: 1.0
f1_score: 1.0
recall_score: 1.0
Accuracy: 1.0
Precision: 1.0
f1_score: 1.0
recall_score: 1.0
```

[65]:
```
print("KNN: Best for local pattern detection")
print("SVM: Best for separating classes")
print("Naive Bayes: Fast and probability-based")
```

```
KNN: Best for local pattern detection
SVM: Best for separating classes
Naive Bayes: Fast and probability-based
```